

Package ‘scanr’

July 2, 2026

Type Package

Title Sequential Change-Point Detection via Nonparametric Inference

Version 0.1.0

Description Detects change points in univariate time series using the SCAN framework. The implementation uses a native Rust backend exposed to R via 'extendr'.

License MIT + file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE)

SystemRequirements Cargo (Rust's package manager), rustc

Imports jsonlite

Config/rextendr/version 0.4.2

NeedsCompilation yes

Config/roxygen2/version 8.0.0

Author Ashoka [aut, cre]

Maintainer Ashoka <smashoka123@gmail.com>

Archs x64

Contents

covering_metric	2
cpd_metrics	2
f1_score_cpd	3
ipm_statistic	3
match_change_points	4
plot_change_points	4
plot_swal_curve	5
plot_thresholds	6
plot_time_series	6
plot_vote_scree	7
plot_window_votes	8
precision_recall_cpd	8
refine_cusum	9
refine_wasserstein	9
scan_cpd	10
scan_single_window	11
swal_statistic	12
wasserstein_statistic	12

Index**13**

covering_metric	<i>Segment covering metric</i>
-----------------	--------------------------------

Description

Computes a weighted segment-covering score in $[0, 1]$.

Usage

```
covering_metric(true_cps, estimated_cps, n)
```

Arguments

true_cps	Integer vector of true change points.
estimated_cps	Integer vector of estimated change points.
n	Number of observations in the series.

Value

Numeric covering score.

cpd_metrics	<i>Combined change-point accuracy metrics</i>
-------------	---

Description

Combined change-point accuracy metrics

Usage

```
cpd_metrics(true_cps, estimated_cps, n, tolerance = 10L)
```

Arguments

true_cps	Integer vector of true change points.
estimated_cps	Integer vector of estimated change points.
n	Number of observations in the series.
tolerance	Maximum absolute distance allowed for a tolerant match.

Value

A list with matches, precision, recall, F1, and covering score.

f1_score_cpd	<i>Tolerant F1 score for change-point detection</i>
--------------	---

Description

Tolerant F1 score for change-point detection

Usage

```
f1_score_cpd(true_cps, estimated_cps, tolerance = 10L)
```

Arguments

true_cps	Integer vector of true change points.
estimated_cps	Integer vector of estimated change points.
tolerance	Maximum absolute distance allowed for a match.

Value

Numeric F1 score.

ipm_statistic	<i>Integral probability metric statistic</i>
---------------	--

Description

Integral probability metric statistic

Usage

```
ipm_statistic(left, right)
```

Arguments

left	Numeric vector.
right	Numeric vector.

Value

Numeric distance.

match_change_points	<i>Match true and estimated change points</i>
---------------------	---

Description

Greedly matches true and estimated change points by smallest distance, allowing each point to be used at most once.

Usage

```
match_change_points(true_cps, estimated_cps, tolerance = 10L)
```

Arguments

true_cps	Integer vector of true change points.
estimated_cps	Integer vector of estimated change points.
tolerance	Maximum absolute distance allowed for a match.

Value

A data frame with columns true, estimated, and distance.

plot_change_points	<i>Plot detected change points from a scan result</i>
--------------------	---

Description

Plot detected change points from a scan result

Usage

```
plot_change_points(
  x,
  result,
  true_change_points = NULL,
  index = NULL,
  x_label = "Time",
  y_label = "Series",
  title = NULL,
  ...
)
```

Arguments

x	Numeric vector.
result	A scanr_result object.
true_change_points	Optional true change points.
index	Optional x-axis index with the same length as x.
x_label	X-axis label.
y_label	Y-axis label.
title	Plot title.
...	Additional arguments passed to plot().

Value

Invisibly returns the plotted data and mapped change points.

plot_swal_curve	<i>Plot the SWAL/Wasserstein localization curve for a region</i>
-----------------	--

Description

Plot the SWAL/Wasserstein localization curve for a region

Usage

```
plot_swal_curve(
  x,
  start,
  end,
  x_label = "Time series",
  y_label = "Scaled Wasserstein statistic",
  title = NULL,
  ...
)
```

Arguments

x	Numeric vector.
start	First observation in the region, using R's one-based indexing.
end	Last observation in the region, inclusive.
x_label	X-axis label.
y_label	Y-axis label.
title	Plot title.
...	Additional arguments passed to plot().

Value

Invisibly returns the curve data and localized change point.

plot_thresholds	<i>Plot scan statistics and bootstrap thresholds</i>
-----------------	--

Description

Plot scan statistics and bootstrap thresholds

Usage

```
plot_thresholds(
  result,
  window_size = NULL,
  x_label = "Window start",
  y_label = "Statistic",
  title = NULL,
  ...
)
```

Arguments

result	A scanr_result object.
window_size	Optional window size to plot. Defaults to the first available window.
x_label	X-axis label.
y_label	Y-axis label.
title	Plot title.
...	Additional arguments passed to plot().

Value

Invisibly returns the threshold data.

plot_time_series	<i>Plot a time series with optional change-point markers</i>
------------------	--

Description

Plot a time series with optional change-point markers

Usage

```
plot_time_series(
  x,
  change_points = NULL,
  true_change_points = NULL,
  index = NULL,
  x_label = "Time",
  y_label = "Value",
  title = "Time series",
  ...
)
```

Arguments

<code>x</code>	Numeric vector.
<code>change_points</code>	Optional detected change points.
<code>true_change_points</code>	Optional true change points.
<code>index</code>	Optional x-axis index with the same length as <code>x</code> .
<code>x_label</code>	X-axis label.
<code>y_label</code>	Y-axis label.
<code>title</code>	Plot title.
<code>...</code>	Additional arguments passed to <code>plot()</code> .

Value

Invisibly returns the plotted data and mapped change points.

<code>plot_vote_scee</code>	<i>Plot retained change-point count by voting threshold</i>
-----------------------------	---

Description

Plot retained change-point count by voting threshold

Usage

```
plot_vote_scee(
  result,
  x_label = "Voting threshold",
  y_label = "Number of retained change points",
  title = NULL,
  ...
)
```

Arguments

<code>result</code>	A <code>scanr_result</code> object.
<code>x_label</code>	X-axis label.
<code>y_label</code>	Y-axis label.
<code>title</code>	Plot title.
<code>...</code>	Additional arguments passed to <code>plot()</code> .

Value

Invisibly returns the scree data.

plot_window_votes	<i>Plot ensemble vote counts for candidate change points</i>
-------------------	--

Description

Plot ensemble vote counts for candidate change points

Usage

```
plot_window_votes(
  result,
  x_label_angle = 45,
  x_label = "Candidate change point",
  y_label = "Window votes",
  title = NULL,
  ...
)
```

Arguments

result	A scanr_result object.
x_label_angle	Rotation angle for x-axis labels.
x_label	X-axis label.
y_label	Y-axis label.
title	Plot title.
...	Additional arguments passed to plot().

Value

Invisibly returns the vote data.

precision_recall_cpd	<i>Tolerant precision and recall for change-point detection</i>
----------------------	---

Description

Tolerant precision and recall for change-point detection

Usage

```
precision_recall_cpd(true_cps, estimated_cps, tolerance = 10L)
```

Arguments

true_cps	Integer vector of true change points.
estimated_cps	Integer vector of estimated change points.
tolerance	Maximum absolute distance allowed for a match.

Value

Named numeric vector with precision and recall.

refine_cusum	<i>Localize a mean change with a CUSUM statistic</i>
--------------	--

Description

Localize a mean change with a CUSUM statistic

Usage

```
refine_cusum(x)
```

Arguments

x Numeric vector.

Value

Integer split position.

refine_wasserstein	<i>Localize a distributional change with a Wasserstein statistic</i>
--------------------	--

Description

Localize a distributional change with a Wasserstein statistic

Usage

```
refine_wasserstein(x)
```

Arguments

x Numeric vector.

Value

A list with change_point and statistics.

scan_cpd

*Detect change points in a univariate time series***Description**

Detect change points in a univariate time series

Usage

```
scan_cpd(
  x,
  window_sizes = NULL,
  alpha = 0.05,
  n_boot = 400L,
  vote_threshold = 0.5,
  min_window = 15L,
  max_window = NULL,
  block_length = NULL,
  taper = c("tukey", "none"),
  tolerance = NULL,
  random_state = NULL,
  n_jobs = NULL,
  return_all = TRUE,
  change_type = c("distribution", "mean", "var"),
  eps = 1e-12,
  batch_size = 32L
)
```

Arguments

x	Numeric vector.
window_sizes	Optional positive integer vector of scan window sizes.
alpha	Significance level, either as a proportion such as 0.05 or a percentage such as 5.
n_boot	Number of tapered block bootstrap replications.
vote_threshold	Minimum normalized ensemble vote score for retained change points.
min_window	Minimum default window size when window_sizes is NULL.
max_window	Maximum default window size when window_sizes is NULL.
block_length	Optional tapered block bootstrap block length.
taper	Taper shape, either "tukey" or "none".
tolerance	Distance used to merge nearby candidates across windows.
random_state	Optional non-negative integer seed.
n_jobs	Optional number of Rust/Rayon worker threads. Defaults to 1. Use -1 for all detected cores.
return_all	Whether to keep per-window diagnostics and raw output.
change_type	One of "distribution", "mean", or "var".
eps	Small positive value used to avoid division by zero.
batch_size	Bootstrap batch size used by the Rust backend.

Value

An object of class `scanr_result`.

<code>scan_single_window</code>	<i>Run SCAN for one window size</i>
---------------------------------	-------------------------------------

Description

Run SCAN for one window size

Usage

```
scan_single_window(
  x,
  window_size,
  alpha = 0.05,
  n_boot = 400L,
  block_length = NULL,
  taper = c("tukey", "none"),
  random_state = NULL,
  change_type = c("distribution", "mean", "var"),
  eps = 1e-12,
  batch_size = 32L
)
```

Arguments

<code>x</code>	Numeric vector.
<code>window_size</code>	Positive integer scan window size.
<code>alpha</code>	Significance level, either as a proportion such as <code>0.05</code> or a percentage such as <code>5</code> .
<code>n_boot</code>	Number of tapered block bootstrap replications.
<code>block_length</code>	Optional tapered block bootstrap block length.
<code>taper</code>	Taper shape, either <code>"tukey"</code> or <code>"none"</code> .
<code>random_state</code>	Optional non-negative integer seed.
<code>change_type</code>	One of <code>"distribution"</code> , <code>"mean"</code> , or <code>"var"</code> .
<code>eps</code>	Small positive value used to avoid division by zero.
<code>batch_size</code>	Bootstrap batch size used by the Rust backend.

Value

An object of class `scanr_window_result`.

swal_statistic	<i>Local SCAN/Wasserstein split statistic</i>
----------------	---

Description

Local SCAN/Wasserstein split statistic

Usage

```
swal_statistic(x, change_type = c("distribution", "mean", "var"))
```

Arguments

x	Numeric vector.
change_type	One of "distribution", "mean", or "var".

Value

Integer split position.

wasserstein_statistic	<i>One-dimensional Wasserstein distance</i>
-----------------------	---

Description

One-dimensional Wasserstein distance

Usage

```
wasserstein_statistic(left, right)
```

Arguments

left	Numeric vector.
right	Numeric vector.

Value

Numeric distance.

Index

covering_metric, [2](#)
cpd_metrics, [2](#)

f1_score_cpd, [3](#)

ipm_statistic, [3](#)

match_change_points, [4](#)

plot_change_points, [4](#)
plot_swal_curve, [5](#)
plot_thresholds, [6](#)
plot_time_series, [6](#)
plot_vote_scree, [7](#)
plot_window_votes, [8](#)
precision_recall_cpd, [8](#)

refine_cusum, [9](#)
refine_wasserstein, [9](#)

scan_cpd, [10](#)
scan_single_window, [11](#)
swal_statistic, [12](#)

wasserstein_statistic, [12](#)